Relatório 27 - Prática: Modelos Generativos (III)

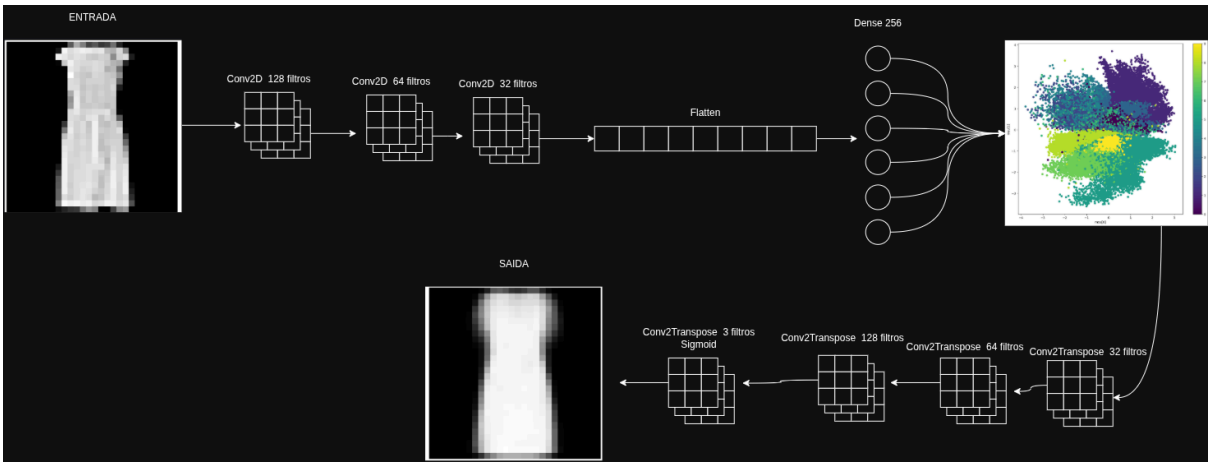
Matheus Beuren Carvalho

1. Introdução

O desenvolvimento do card se resume no aprendizado do conteúdo de modelos generativos. Onde este relatório abordará modelos que conseguem gerar imagens a partir de seu treino e geração de texto, onde é introduzido a arquitetura GPT. Nisso, ao longo das aulas foi realizada a prática de um modelo pré-treinado e depois aplicado um fine-tuning.

2. Desenvolvimento

Os AutoEncoders são uma maneira simplificada de destilar uma imagem e remontá-la. Isso é feito através de um encoder e decoder, por exemplo, uma imagem entra e o encoder faz a modificação para que ela fique em um espaço latente(um espaço compactado) e depois com o decoder ela é remontada. Nesse contexto, ela é uma maneira diferente de comparar com uma CNN, mas ela estuda o que realmente define um sapato, o que é diferente de um para outro. Com base nessa perspectiva de característica central, ele pode ser utilizado para gerar novas imagens sintéticas. Na figura 1, pode ser observado como funciona um auto encoder.



Os VAEs(Variational Auto Encoder) são auto encoders como explicados anteriormente, porém ele se baseia na distribuição de probabilidade no espaço latente, nisso o ponto que será usado é uma amostra desta distribuição. Nesse contexto, existem duas funções de perda(loss), o “Reconstruction loss” que mede o quão bem a saída do decoder se parece com a entrada original, e também o “KL DIvergence” que força a distribuição no espaço latente a se aproximar da média e do desvio padrão.

Para realizar a prática de VAEs, foi utilizado o CIFAR-10 dataset grande e complexo. No notebook, pode-se observar um resultado muito abaixo ao comparar com o MINIST feito em aula, as imagens geradas pelo espaço latente após o treino foram bem ruins. Isso se justifica, pela alta complexidade do CIFAR-10, onde contém muitas classes e o principal fator é a quantidade de canais onde as imagens do cifar são rgb enquanto as do minist estão em escala de cinza.

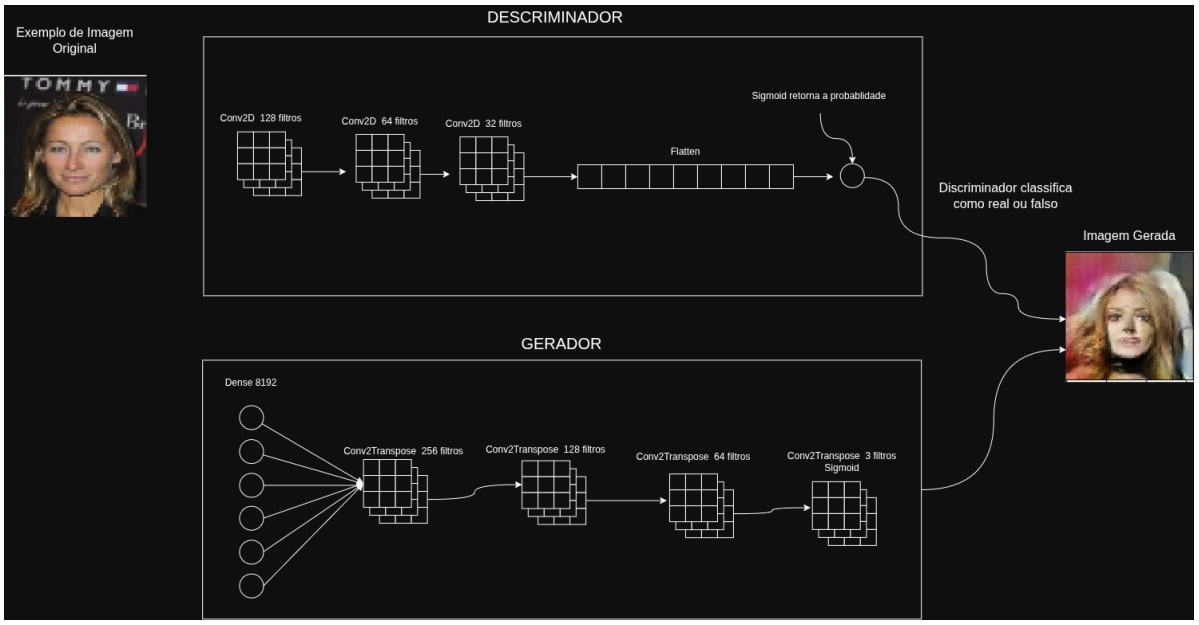
As GANs é um modelo generativo onde duas redes competem entre si para gerar uma imagem. Algumas de suas aplicações é seu uso para sites que preveem seu envelhecimento, deepfakes, em obras de arte e até geração de um dataset de rostos para o treino de um detector de faces.

Diferente dos VAEs, ao utilizar GANs será usado duas redes chamadas de ‘Gerador’ e ‘Discriminador’, onde uma vai receber um ruído aleatório e aprende a criar imagens, e o outra vai analisar as imagens reais e as geradas para aprender a distinguir a real da falsa. O ponto central é que o gerador tenta enganar o discriminador para que não reconheça a diferença entre as duas imagens e o discriminador vai tentar pegar o gerador criando imagens falsas, terminando o treino quando o discriminador não consegue mais diferenciar as imagens reais das geradas.

Para fins didáticos, foi apresentado o ‘GAN Lab’ que simula um GAN em seu aprendizado e já foi treinado. No vídeo relacionado a GANs, podemos ver que a partir de um desenho simples é gerado uma imagem por meio das características aprendidas no treino da GAN.

Em vista de realizar a prática do conteúdo de GANs, utilizei um dataset de rostos, onde as imagens são maiores comparadas às de aula(64x64) e estão em rgb. E em meio aos experimentos a fim de encontrar um bom modelo, foi realizado uma pesquisa na documentação do tensorflow e achado um parâmetro que pode ajudar a controlar como as redes aprendem,restringindo sua norma espectral(TENSORFLOW). O motivo desta escolha se resume pela velocidade de aprendizado do discriminador, com ‘SpectralNormalization’ os pesos de uma camada ficam restritos a uma magnitude controlada, assim evitando que o discriminador fique confiante muito rápido. Na figura 2, é mostrado como uma GAN funciona.

Figura 2 - GAN



Um modelo generativo chamado GPT, mostrado no segundo vídeo, calcula as probabilidades e por isso não é um modelo determinístico onde sempre é retornada a mesma mensagem, dado o mesmo texto de entrada.

Ao decorrer do vídeo de modelos generativos GPT, foi apresentado e ensinado um conteúdo não visto antes, um modelo que gera texto. Nada mais é do que um cálculo de probabilidades, onde dado tokens passados o objetivo é prever o próximo token a partir do que já foi observado.

Nesse contexto, a seguinte aula demonstrou como é feito desde o processamento do texto(NLP) até a previsão de novas palavras e assim ao final gerando texto completo. O modelo aprende conforme o seu contexto anterior(palavras passadas), onde o contexto influencia diretamente na previsão.

Self-attention é o conteúdo principal do vídeo, onde ela permite que uma determinada posição de uma sequência possa ser representada considerando os tokens anteriores. Outro ponto, é a afinidade representada em tokens na mesma sequência, um token na posição atual calcula um peso de quanto cada token anterior é relevante para ele prever um próximo caractere.

Key é uma função linear aplicada a um vetor de embedding que ao comparar com Query(uma representação do que um token precisa), constrói-se uma afinidade que é utilizada para dar representar uma coerência entre os tokens. Essa afinidade pondera os values, gerando a representação contextualizada, gerando um próximo token que seja coerente aos tokens anteriores.

A máscara casual serve para que o modelo não roube no treino, sabendo antes de prever qual é o próximo token, para resolver isso cria-se uma matriz que permite o modelo enxergar somente o que já passou. O multi-head, são vários headers procurando afinidades no texto, permitindo com que o modelo aprenda diferentes tipos de afinidade ao mesmo tempo, ao final tudo é concatenado em uma única camada linear.

O vídeo em questão refaz a arquitetura de um modelo generativo com base no artigo “Attention is All You Need” que aborda como é feito o modelo GPT. Nesse sentido, utilizei a mesma arquitetura para realizar a prática do conteúdo, porém aplicando o pré-learning com um dataset do Hugging Face sobre artigos brasileiros, e com a intenção de ver como o modelo se comporta no fine-tuning com uma base de obras de machado de assis. O desempenho do modelo foi consideravelmente bom, porém não obteve tanta eficácia ao treinar e utilizar o modelo. O principal motivo para não ter um resultado bom, é o tempo de treinamento e de computação, mesmo ao utilizar o google colab, tive que limitar os caracteres da base do wikipedia, e iterações de treino. Para obter melhor resultado com esta prática, seria necessário mais tempo de treinamento.

3. Conclusão

Os modelos generativos apresentados no card são diferentes tipos de geração como imagens e texto. Nesse contexto, as abordagens aprendidas podem ser utilizadas para criar datasets de imagens sintéticas e textos. A principal utilização da geração de texto, hoje conhecido como LLM(large language model) é muito utilizado para escrita e bate papo se assemelhando a um humano. A criação de um modelo a partir de uma base do wikipedia, se mostrou extremamente rico em conteúdo, porém generalista. Ao final do pré-learning(treino), o modelo não alcançou seu desempenho máximo, por conta do tempo de processamento necessário, porém essa limitação pode ser superada ao aumentar o loop de treino. Ao realizar o fine-tuning observei que o modelo não conseguiu obter bons resultados, já que os textos de Machado de Assis são de alto nível literário.

4. Referências

TENSORFLOW V2.16.1. [*S. l.: s. n.*], [*s. d.*]. Disponível em: https://www.tensorflow.org/api_docs/python/tf/keras/layers/SpectralNormalization. Acesso em: 22 set. 2026.